



«Разработка веб-интерфейсного скрипта»

Система контроля версий. Основы работы с git и GitHub.

Преподаватель кафедры ИСПИ
Хлызова Валерия Григорьевна.

Любое веб-приложение состоит из множества файлов, которые структурированы в проекте. Самая простая веб-страница состоит из нескольких элементов HTML + CSS + JavaScript.

HTML — это язык разметки, который используется для создания структуры веб-страниц. Он состоит из тегов, которые обозначают различные элементы страницы, такие как заголовки, абзацы, списки, ссылки и множество других элементов.

```
<!DOCTYPE HTML>  
<html>  
  <body>  
    <form>  
      <textarea cols="20" rows="10"></textarea>  
    </form>  
  </body>  
</html>
```

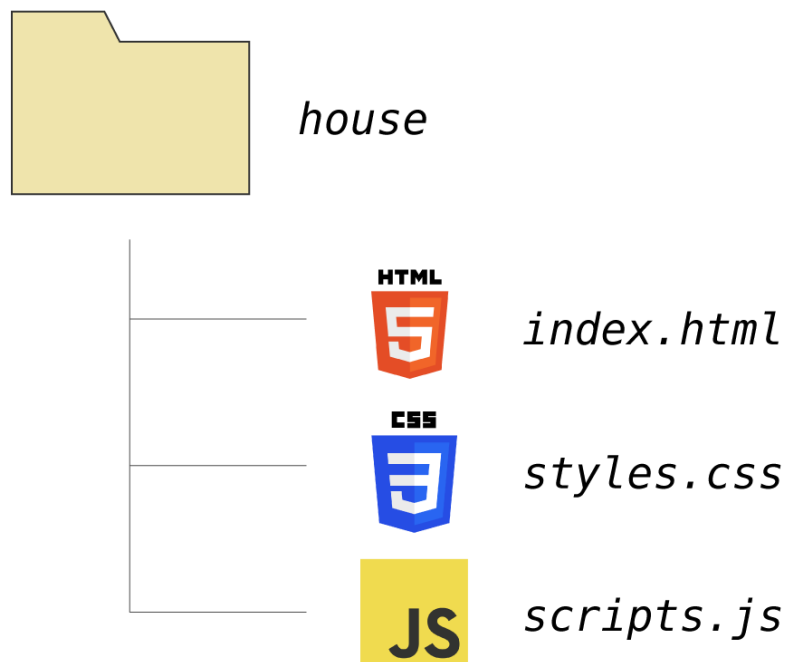
CSS — это язык стилей, используемый для описания внешнего вида веб-страницы. Он позволяет задавать различные стили для HTML-элементов, такие как цвет, размер шрифта, отступы и многое другое. CSS-стили можно применять к HTML-элементам с помощью селекторов.

```
.light {  
  width: 100px;  
  height: 100px;  
  background-color: black;  
  margin: 10px;  
  border-radius: 50%;  
}
```

JavaScript — это язык программирования, который используется для добавления интерактивности, анимации и обработки пользовательских действий на веб-страницах. JavaScript можно встраивать в HTML-документы или подключать как отдельные файлы.

```
function turnOn() {  
    document.getElementById('light').style.backgroundColor = 'yellow';  
    document.getElementById('light').style.boxShadow = "0px 0px 50px yellow";  
}  
  
function turnOff() {  
    document.getElementById('light').style.backgroundColor = 'black';  
    document.getElementById('light').style.boxShadow = "none";  
}
```

HTML, CSS и JavaScript работают вместе, чтобы создать динамичные и красивые веб-страницы. HTML определяет структуру и содержимое страницы, CSS задает стили и оформление, а JavaScript добавляет интерактивность и дополнительные функции.



Реализацию этих трех компонентов (HTML, CSS и JavaScript) можно выполнить в одном файле. Но на практике их выносят в отдельные с соответствующим расширением.

Для одной веб-страницы может использоваться несколько CSS и JavaScript файлов, а весь проект (даже небольшого размера) может содержать около ста различных файлов.

Здесь разработчику приходит на помощь система контроля версий и репозиторий GitHub.

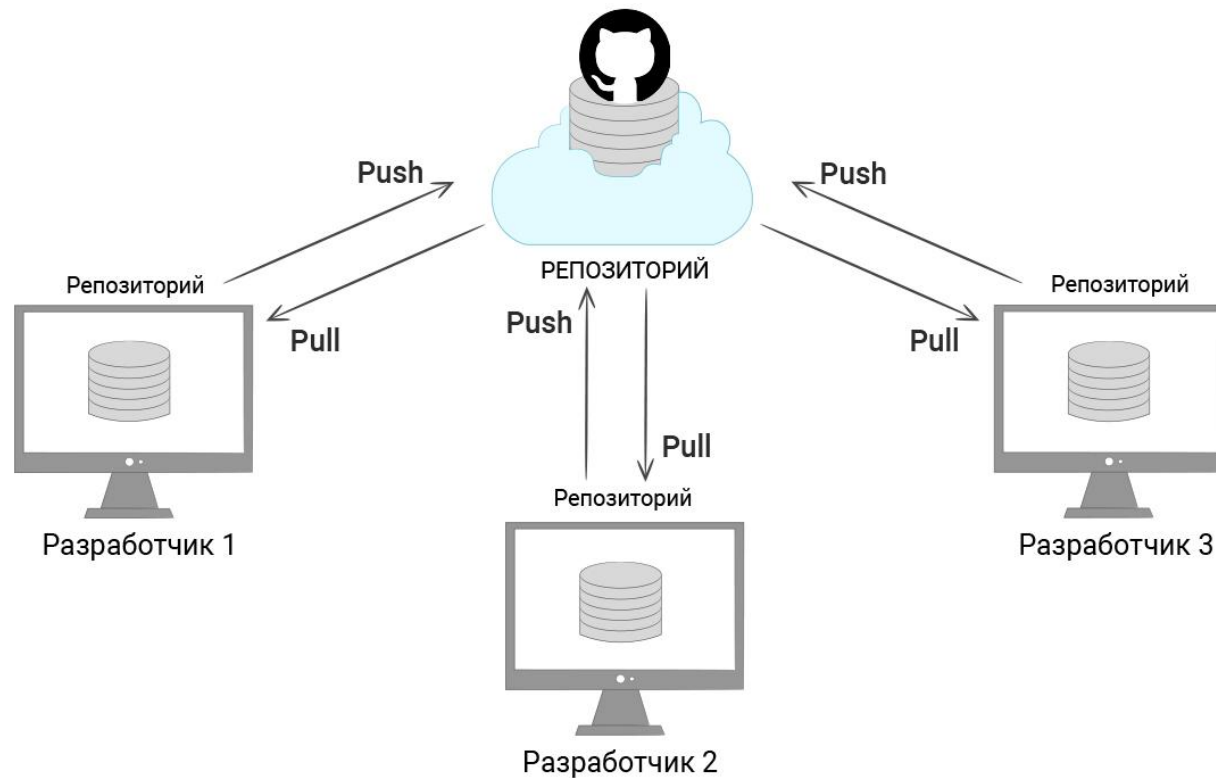
Система контроля версий — это система, записывающая изменения в файл или набор файлов в течение времени и позволяющая вернуться позже к определённой версии.

git — самый популярный вариант системы контроля версий, ставший синонимом «управления исходным кодом». Это консольная утилита, для отслеживания и ведения истории изменения файлов, в проекте. У **git** две основных задачи: первая — хранить информацию о всех изменениях в коде, начиная с самой первой строчки, а вторая — обеспечение удобства работы над кодом и доступ к коду.

Все изменения Вашего кода фиксируются в локальном репозитории на вашем компьютере, а затем отправляются в удаленный репозиторий для сохранности. Каждому разработчику необходимо уметь работать с локальными и удалёнными репозиториями.

*Репозиторий — место, где хранятся и поддерживаются какие-либо данные. В нашем случае файлы веб-приложения.

Удалённые репозитории представляют собой версии проекта, сохранённые в интернете или ещё где-то в сети. Разработчики могут фиксировать локальные изменения кода и отправлять их в удаленный репозиторий, или наоборот, загружать изменения из удаленного репозитория на свой компьютер.

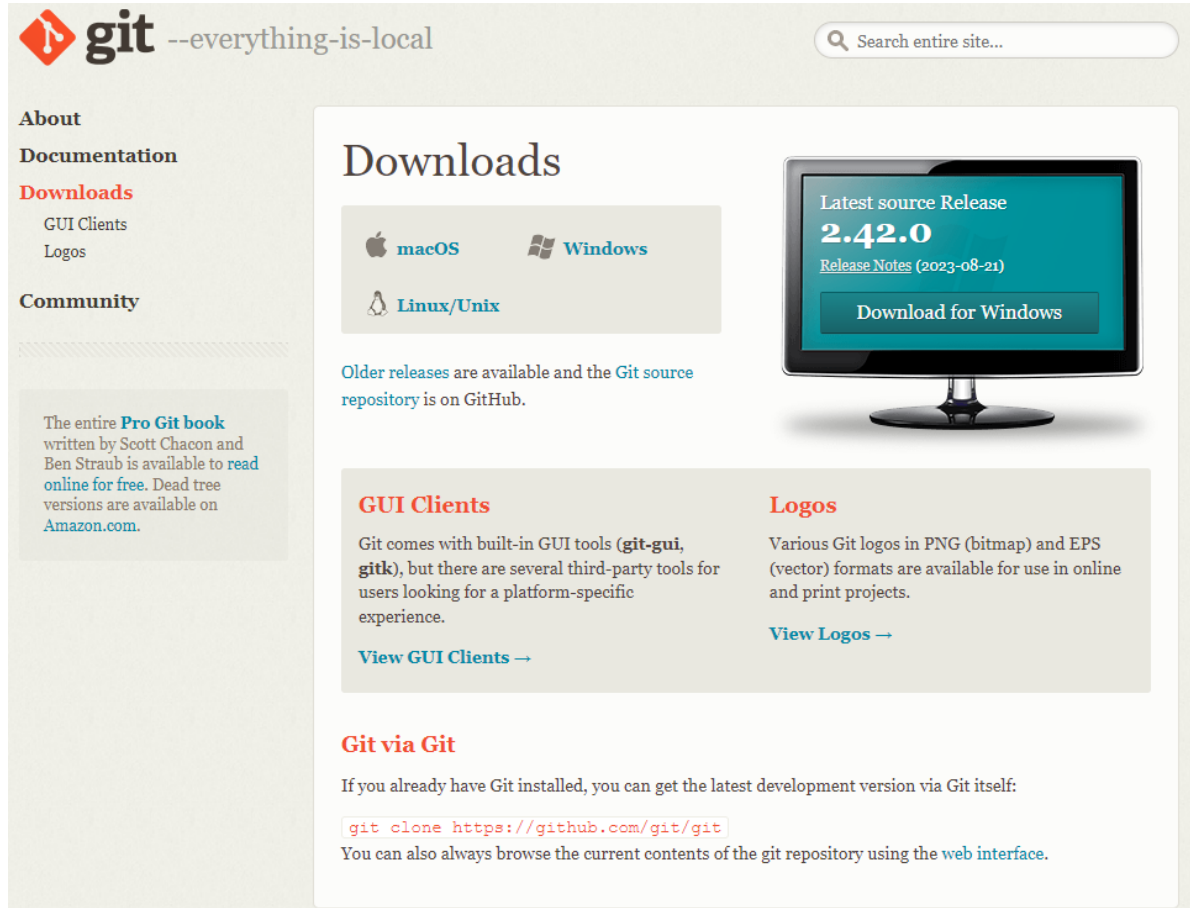


 **GitHub** — крупнейший веб-сервис для размещения IT-проектов. Веб-сервис основан на системе контроля версий **git**. Веб-сервис служит для создания удаленных репозиториев.

Используя **Git** и **GitHub** Вы никогда не потеряете свои изменения. Используя данный сервис Ваш код всегда будет у Вас под рукой. Так вы сможете работать на разном оборудовании, и доступ к коду можно будет осуществить даже с мобильного устройства. Вы сможете работать над кодом вместе со своими одногруппниками и преподавателями.

Разберем шаг за шагом как начать работать с **git** и **GitHub**.

Чтобы использовать **git**, разработчику необходимо
загрузить, установить и настроить **git** на своем
компьютере.



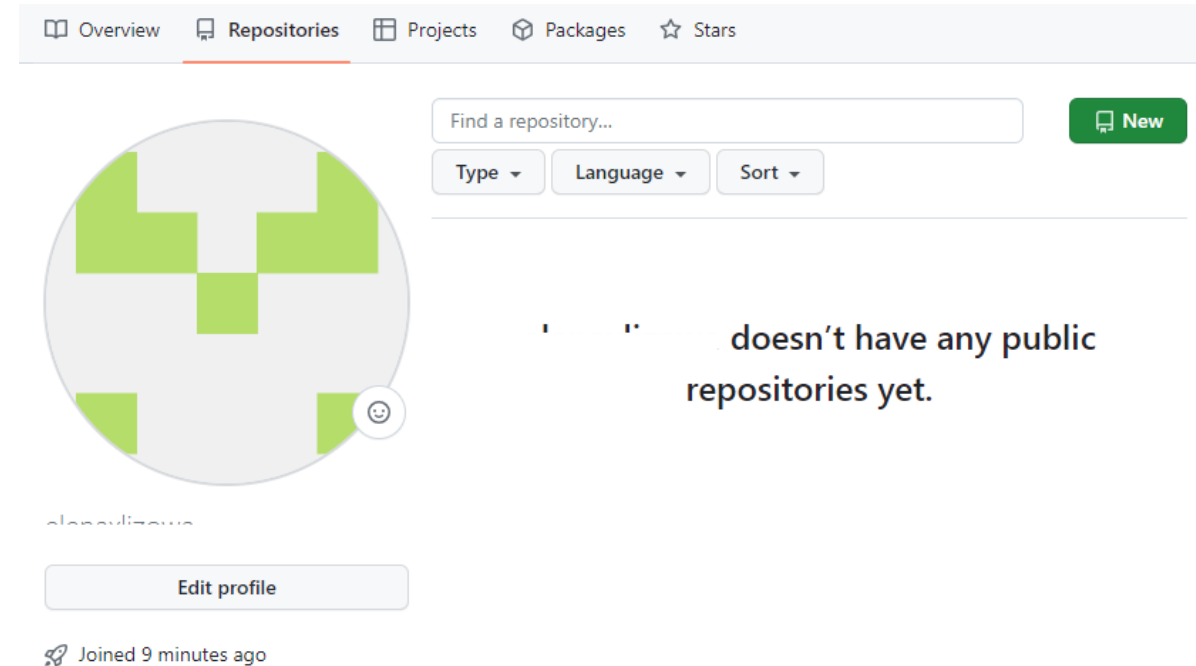
- Установить git можно с официального сайта: <https://git-scm.com/>
- После загрузки систему контроля версий можно использовать через **командную строку ОС** или **GitBash** (Терминал для выполнения команд git).

Чтобы проверить правильно ли прошла установка можно выполнить команду **git version** в терминале. Если установка прошла успешна в терминале отобразится текущая версия системы контроля версий **git**.

```
valer@DESKTOP-IF1SD4B MINGW64 ~  
$ git version  
git version 2.42.0.windows.2
```

Чтобы использовать возможности **GitHub**, в нем необходимо зарегистрироваться. Для регистрации понадобится только адрес электронной почты.

После регистрации перейдите в раздел **Repositories** и создайте новый репозиторий.



Для создания нового репозитория необходимо указать имя репозитория и его краткое описание.

Вы можете сделать репозиторий общедоступным, указав опцию Public при создании — все пользователи **GitHub** будут видеть Ваш код. Или сделать его приватным, выбрав опцию Private — только Вы будете видеть Ваш код.

Так же при инициализации репозитория добавьте файл README.

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner *	Repository name *
 . . .	/ JavaScript
	✔ JavaScript is available.

Great repository names are short and memorable. Need inspiration? How about [musical-octo-pancake](#) ?

Description (optional)

Репозиторий для практических занятий по дисциплине "Разработка веб-интерфейсного скрипта"

- ☒  **Public**
Anyone on the internet can see this repository. You choose who can commit.
- ☐  **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

- ☒ **Add a README file**
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None ▾

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None ▾

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

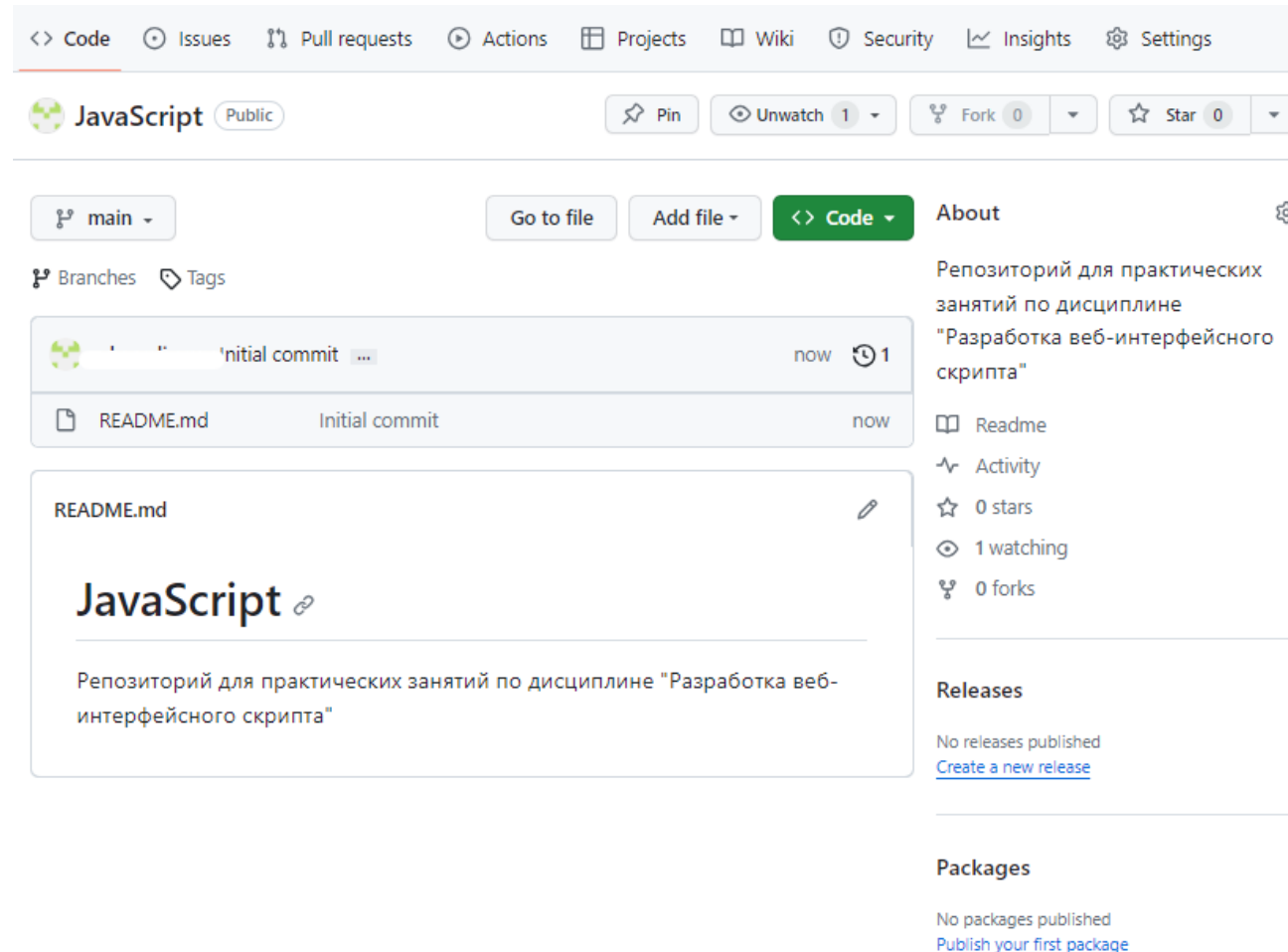
This will set  **main** as the default branch. Change the default name in your [settings](#).

 You are creating a public repository in your personal account.

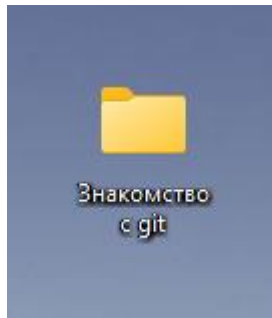
README — текстовый файл, который распространяется вместе с программным обеспечением и содержит информацию о нём. Обычно это информация о продукте, например цель разработки, информация о разработчиках, или инструкция по развертыванию и использованию приложения.

GitHub автоматически сгенерирует содержания файла **README**, расположив в нем название репозитория и его описание.

На данном слайде представлен пример созданного репозитория.
Выполните копирование URL адреса страницы.



Создайте папку на рабочем столе, затем вернитесь в терминал, перейдите к расположению созданной папки и клонируйте созданный на **GitHub** репозиторий на свой компьютер: **git clone <скопированный URL адрес>**



```
valer@DESKTOP-IF15D4B MINGW64 ~/OneDrive/Рабочий стол/Знакомство с git (master)
$ git clone https://github.com/valeriy-ivanov/JavaScript
Cloning into 'JavaScript'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (3/3), done.
```



Знакомство с git > JavaScript			
Имя	Дата изменения	Тип	Размер
 README.md	31.10.2023 21:07	Исходный файл ...	1 КБ

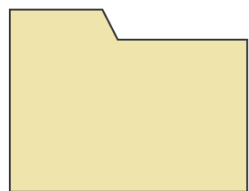
Проверьте, что на компьютере, в созданной папке появился сгенерированный файл **README**. Так же проверьте его содержимое.

Знакомство с git > JavaScript			
Имя	Дата изменения	Тип	Размер
 README.md	31.10.2023 21:07	Исходный файл ...	1 КБ

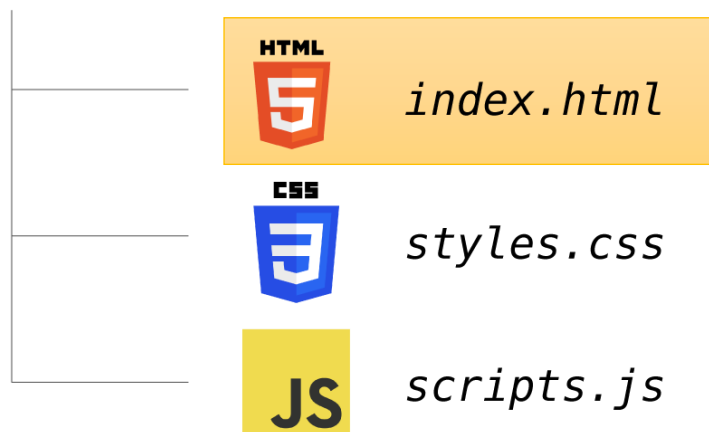
Создадим нашу первую веб страницу и сохраним её в удаленном репозитории. Для этого в рабочей папке создадим новую папку *First program* и добавим в неё три файла: *page.html*, *script.js* и *style.css*.

Знакомство с git > JavaScript >				
Имя	Дата изменения	Тип	Размер	
First program	31.10.2023 21:09	Папка с файлами		
README.md	31.10.2023 21:07	Исходный файл ...	1 КБ	

Знакомство с git > JavaScript > First program				
Имя	Дата изменения	Тип	Размер	
page.html	31.10.2023 21:08	Chrome HTML Do...	0 КБ	
script.js	31.10.2023 21:08	WebStorm2022.1	0 КБ	
style.css	31.10.2023 21:08	WebStorm2022.1	0 КБ	

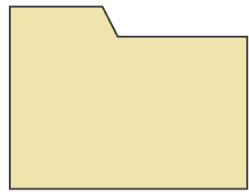


house



Содержание файла HTML

```
<!DOCTYPE HTML>
<html>
<header>
  <meta charset="UTF-8">
  <link href="style.css" rel="stylesheet"
type="text/css" />
</header>
<body>
  <h1>My first program!</h1>
  <script src="script.js"></script>
</body>
</html>
```

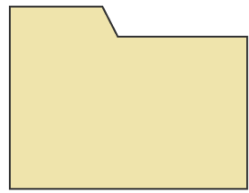


house



Содержание файла CSS

```
h1 {  
    color: green;  
    font-size: large;  
    font-family: Arial, Helvetica, sans-serif;  
}
```



house



index.html



styles.css



scripts.js

Содержание файла JavaScript

```
alert('Hello world!');
```

Теперь все внесенные локальные изменения нужно зафиксировать в удаленном репозитории. Данная фиксация носит название «Коммит».

Коммит - это способ сохранения изменений в коде. Каждый коммит содержит информацию о том, что было изменено в коде и кем были внесены эти изменения. Они позволяют разработчикам отслеживать изменения в своем коде и возвращаться к предыдущим версиям, если это необходимо.

Перед выполнением коммита мы можем проверить текущий статус локального репозитория. Для этого нужно выполнить команду **git status**

На изображении видно что есть незафиксированные изменения **First program/** — название папки с кодом нашего первого веб-приложения.

```
valer@DESKTOP-IF1SD4B MINGW64 ~/OneDrive/Рабочий стол/Знакомство с git/JavaScript
(main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    First program/

nothing added to commit but untracked files present (use "git add" to track)
```

Чтобы использовать команду `commit`, разработчик должен сначала использовать команду **`git add`**, чтобы добавить измененные файлы.

Есть несколько вариантов выполнения данной команды:

`git add <название файла>`

`git add .`

Точка — обозначает все измененные файлы.

```
valer@DESKTOP-IF1SD4B MINGW64 ~/OneDrive/Рабочий стол/Знакомство с git/JavaScript
(main)
$ git add .

valer@DESKTOP-IF1SD4B MINGW64 ~/OneDrive/Рабочий стол/Знакомство с git/JavaScript
(main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   First program/page.html
    new file:   First program/script.js
    new file:   First program/style.css
```

После того как файлы добавлены (**git add .**), разработчик может использовать команду **git commit -m <сообщение>**, чтобы сохранить изменения. При этом нужно написать сообщение о внесенных изменениях. Сообщение обычно должно содержать краткое описание того, что сделал разработчик и что теперь работает по-другому.

```
valer@DESKTOP-IF1SD4B MINGW64 ~/OneDrive/Рабочий стол/Знакомство с git/JavaScript
(main)
$ git commit -m 'my first commit'
[main cb35a7a] my first commit
3 files changed, 17 insertions(+)
create mode 100644 First program/page.html
create mode 100644 First program/script.js
create mode 100644 First program/style.css

valer@DESKTOP-IF1SD4B MINGW64 ~/OneDrive/Рабочий стол/Знакомство с git/JavaScript
(main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

Хорошее сообщение должно содержать краткое описание внесенных изменений и иметь понятную структуру.

- Старайтесь использовать понятные и информативные заголовки.
- Опишите внесенные изменения и поясните, почему они были сделаны.
- Используйте простой язык и старайтесь избегать сложных технических терминов.

Зафиксированные локальные изменения можно перенести в удаленный репозиторий на платформу **GitHub**. Для этого в терминале выполняется команда **git push origin main**

Теперь доступ к Вашему первому веб-сайту можно получить с платформы **GitHub**. Проверьте наличие папки *First program* в Вашем созданном репозитории и содержание файлов.

 main ▾ 1 branch 0 tags

[Go to file](#) [Add file ▾](#) [Code ▾](#)

 my first commit ddf070 4 minutes ago 2 commits

First program	my first commit	4 minutes ago
README.md	Initial commit	1 hour ago

README.md 

JavaScript

Репозиторий для практических занятий по дисциплине "Разработка веб-интерфейсного скрипта"

 Files

 main ▾ + 🔍

First program

- page.html
- script.js
- style.css
- README.md

JavaScript / First program / page.html 

 my first commit ddf070 6 minutes ago History

11 lines (11 loc) • 226 Bytes  Code 55% faster with GitHub Copilot

Code Blame

Raw    

```
1 <!DOCTYPE HTML>
2 <html>
3 <header>
4   <meta charset="UTF-8">
5   <link href="style.css" rel="stylesheet" type="text/css" />
6 </header>
7 <body>
8   <h1>My first program!</h1>
9   <script src="script.js"></script>
10 </body>
11 </html>
```

Для извлечения и загрузки содержимого из удаленного репозитория и немедленного обновления локального репозитория этим содержимым используется команда **git pull**

Это обратный процесс команды **git push**

```
valer@DESKTOP-IF1SD4B MINGW64 .  
t (main)  
$ git pull  
Already up to date.
```

